Геолокация

Содержание лекции



Permissions: Основа всего

Разрешения: Точность и Контекст

1. Выбор Точности (Что мы хотим?):
   - ACCESS_FINE_LOCATION: Высокая точность (метры). Нужна для навигации. Использует GPS.
   - ACCESS_COARSE_LOCATION: Низкая точность (квартал/город). Для погоды, общих рекомендаций. Экономит батарею.
2. Выбор Контекста (Когда мы это хотим?):
   - Foreground (Передний план): Приложение получает геоданные, только когда активно и видимо пользователю.
   - Background (Фоновый режим): Приложение получает геоданные, даже когда свернуто или закрыто.

Объявляем разрешения в AndroidManifest.xml

Какие разрешения вам нужны, необходимо заранее объявить в манифесте.

Для приблизительной точности:

```
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

Объявляем разрешения в AndroidManifest.xml

Какие разрешения вам нужны, необходимо заранее объявить в манифесте.

Для высокой точности (включает в себя и приблизительную):

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Объявляем разрешения в AndroidManifest.xml

Какие разрешения вам нужны, необходимо заранее объявить в манифесте.

Для работы в фоновом режиме (начиная с Android 10):

```
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
```

Запрашиваем разрешения у пользователя

Начиная с Android 6, недостаточно просто объявить разрешения, их нужно запросить у пользователя во время работы приложения.

Лучшая практика: использовать ActivityResultContracts.RequestMultiplePermissions

Запрашиваем разрешения у пользователя

```
// 1. Готовим лаунчер для запроса разрешений
private val locationPermissionRequest = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    // 3. Обрабатываем ответ пользователя
    when {
        permissions.getOrDefault(Manifest.permission.ACCESS_FINE_LOCATION, false) -> {
            // Доступ к точной геолокации предоставлен.
            // Можно запускать логику, требующую высокой точности.
        }
        permissions.getOrDefault(Manifest.permission.ACCESS_COARSE_LOCATION, false) -> {
            // Предоставлен доступ только к приблизительной геолокации.
            // Запускаем альтернативную логику.
        } else -> {
            // Пользователь отказал в доступе.
            // Объясняем, почему фича недоступна.
        }
    }
}
```

Запрашиваем разрешения у пользователя

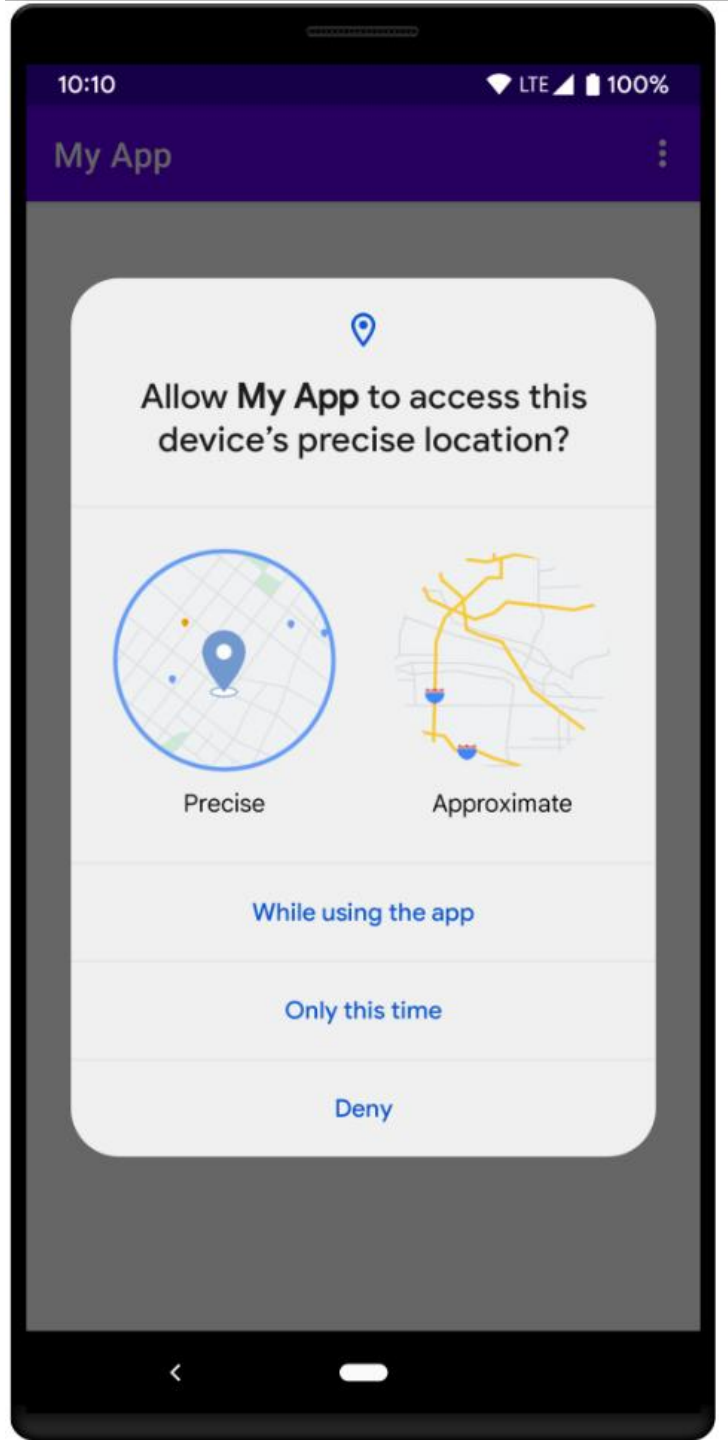
```
// 2. В нужный момент (например, по клику на кнопку) запускаем запрос
fun askForLocationPermissions() {
    locationPermissionRequest.launch(arrayOf(
        Manifest.permission.ACCESS_FINE_LOCATION,
        Manifest.permission.ACCESS_COARSE_LOCATION))
}
```

Что видит пользователь (Android 12+)

Если вы запрашиваете ACCESS_FINE_LOCATION и ACCESS_COARSE_LOCATION вместе, система показывает такой диалог:

1. Выбор точности: Пользователь может выбрать «Точное» или «Приблизительное». Ваше приложение получит только те права, что он выбрал.

2. Выбор времени:
   - «При использовании приложения» (Foreground)
   - «Только в этот раз» (Временное разрешение)
   - «Запретить»

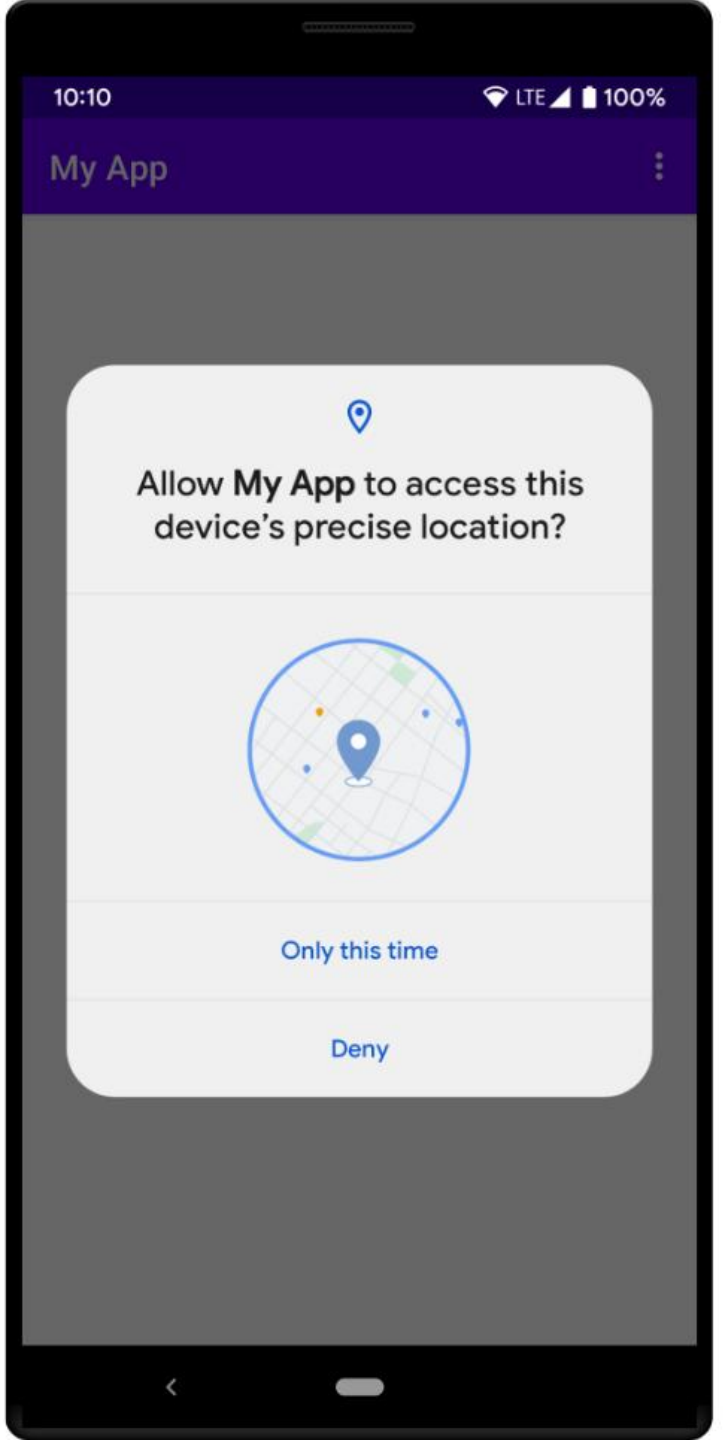


Запрос фонового доступа: Особый случай

Правило №1: Нельзя запрашивать фоновый доступ (ACCESS_BACKGROUND_LOCATION) одновременно с обычным.

Правильная последовательность (инкрементный запрос):

1. Сначала получите разрешение на foreground геолокацию (FINE + COARSE).



Запрос фонового доступа: Особый случай

Правило №1: Нельзя запрашивать фоновый доступ (ACCESS_BACKGROUND_LOCATION) одновременно с обычным.

Правильная последовательность (инкрементный запрос):

2. Когда пользователю понадобится фоновая функция, покажите ему свой собственный экран или диалог, объясняющий, зачем это нужно. Пример: *«Чтобы записывать ваш маршрут, даже когда телефон в кармане, нам нужен доступ к геолокации в фоновом режиме».*

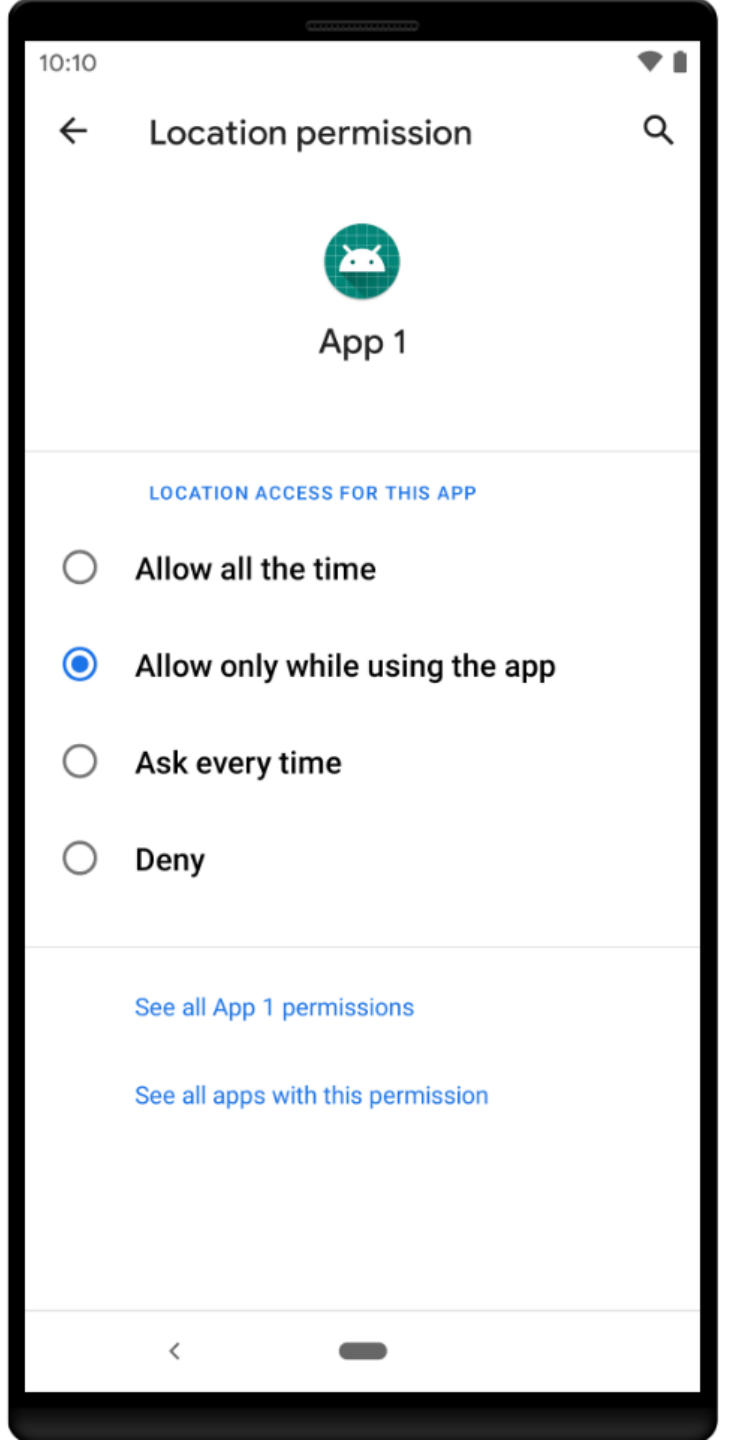
Запрос фонового доступа: Особый случай

Правило №1: Нельзя запрашивать фоновый доступ (ACCESS_BACKGROUND_LOCATION) одновременно с обычным.

Правильная последовательность (инкрементный запрос):

3. Когда пользователю понадобится фоновая функция, покажите ему свой собственный экран или диалог, объясняющий, зачем это нужно. Пример: *«Чтобы записывать ваш маршрут, даже когда телефон в кармане, нам нужен доступ к геолокации в фоновом режиме».*

Важно: На Android 11+ система не покажет диалог, а перенаправит пользователя на экран настроек вашего приложения. Ваша задача — сопроводить его и объяснить, какую опцию выбрать («Разрешить в любом режиме»).



Получение геолокации: 3 главных сценария

Прежде чем начать: Проверка готовности

Перед тем как запрашивать координаты, нужно убедиться в двух вещах:

1. Google Play Services доступны?
   - Все Location API являются их частью. Если сервисов нет, приложение упадет.
   - Решение: Добавьте проверку в onCreate вашей Activity.

```
val googleApiAvailability = GoogleApiAvailability.getInstance()
val resultCode = googleApiAvailability.isGooglePlayServicesAvailable(this)
if (resultCode != ConnectionResult.SUCCESS) {
    googleApiAvailability.getErrorDialog(this, resultCode, 0)?.show()
    // Дальнейшая логика геолокации не должна выполняться
}
```

Прежде чем начать: Проверка готовности

Перед тем как запрашивать координаты, нужно убедиться в двух вещах:

2. Геолокация включена в настройках устройства?
    - Если GPS выключен, запросы не дадут результата.
    - Решение: Проверьте настройки перед запросом и предложите пользователю их включить. (Код для этого мы рассмотрим вместе с requestLocationUpdates)

Способ 1: getLastLocation()

Когда использовать: Для быстрых, однократных запросов, где не нужна точность «до секунды» (например, погода, поиск «кафе рядом»).

Способ 1: getLastLocation()

Как это работает:

1. Инициализируем клиент:

```
private lateinit var fusedLocationClient: FusedLocationProviderClient
// ... в onCreate()
fusedLocationClient = LocationServices.getFusedLocationProviderClient(this)
```

Способ 1: getLastLocation()

Как это работает:

2. Запрашиваем последнее местоположение:

```
fusedLocationClient.lastLocation
    .addOnSuccessListener { location: Location? ->
        // Результат может быть null!
        if (location != null) {
            // Логика с использованием координат
            Log.d("MyLog", "Lat: ${location.latitude}, Lon: ${location.longitude}")
        } else {
            // Координаты неизвестны. Обрабатываем эту ситуацию.
            Log.d("MyLog", "Location is null")
        }
    }
```

Способ 1: getLastLocation()

ВАЖНО: Результат может быть null!

- Геолокация отключена на устройстве.
- Устройство новое или после сброса настроек.

Способ 2: requestLocationUpdates()

Когда использовать: Для постоянного отслеживания в реальном времени (навигация, фитнес-трекеры).

Способ 2: requestLocationUpdates()

Процесс:

1. Создаем LocationRequest (детально разберем на следующем слайде).
2. Создаем LocationCallback для получения результатов.

```
private lateinit var locationCallback: LocationCallback
// ... в onCreate()
locationCallback = object : LocationCallback() {
    override fun onLocationResult(locationResult: LocationResult) {
        for (location in locationResult.locations){
            // Обновляем UI новыми координатами
        }
    }
}
```

Способ 2: requestLocationUpdates()

Как это работает:

3. Запускаем и останавливаем обновления в соответствии с жизненным циклом!

```
override fun onResume() {
    super.onResume()
    startLocationUpdates()
}
private fun startLocationUpdates() {
    fusedLocationClient.requestLocationUpdates(locationRequest,
        locationCallback, Looper.getMainLooper())
}
override fun onPause() {
    super.onPause()
    stopLocationUpdates()
}
private fun stopLocationUpdates() {
    fusedLocationClient.removeLocationUpdates(locationCallback)
}
```

Настройка LocationRequest: Управляем батареей

LocationRequest — это объект, который описывает, КАК мы хотим получать обновления.

```
private fun createLocationRequest(): LocationRequest {
    return LocationRequest.create().apply {
        // 1. Как часто мы хотим обновления
        interval = 10000 // 10 секунд

        // 2. Как часто мы готовы получать их в самом быстром случае
        fastestInterval = 5000 // 5 секунд

        // 3. Какую точность мы хотим (главный параметр!)
        priority = LocationRequest.PRIORITY_HIGH_ACCURACY
    }
}
```

Настройка LocationRequest: Управляем батареей

priority — ключ к экономии батареи:

- PRIORITY_HIGH_ACCURACY: Максимальная точность (GPS). Для навигаторов. Очень затратно.
- PRIORITY_BALANCED_POWER_ACCURACY: Точность на уровне квартала (~100м). Хороший баланс для большинства задач.
- PRIORITY_LOW_POWER: Точность на уровне города. Низкое потребление.
- PRIORITY_NO_POWER: Пассивный режим. Не делает запрос, но получает данные, если другое приложение их запросило.

Способ 3: Geofencing API — виртуальные границы

Когда использовать: Когда нужно отреагировать на вход/выход из определенной зоны, а не отслеживать каждый шаг.

Примеры: «Напомнить купить молоко у магазина», «Автоматически включать Wi-Fi дома».

Как работает (концептуально):

1. Вы создаете «геозону» (Geofence): круг с центром (широта, долгота) и радиусом.
2. Регистрируете ее в системе и «забываете» про нее.
3. Система сама, в фоновом режиме и с минимальным расходом батареи, отслеживает пересечение границы.
4. Когда событие происходит (вход/выход), ваше приложение получает уведомление через BroadcastReceiver.

Способ 3: Geofencing API — виртуальные границы

Главный плюс: Не нужно держать приложение запущенным и постоянно запрашивать координаты. Система делает это за вас максимально эффективно.

Требует разрешения: ACCESS_BACKGROUND_LOCATION.

Оптимизация: Как не разрядить батарею

Три кита расхода батареи

Сбор данных о местоположении — одна из самых энергозатратных операций. Мы управляем расходом через три параметра:

1. Точность (Priority):
   - Чем выше точность (GPS), тем больше энергии тратится на работу радиомодуля.

2. Частота (Interval):
   - Чем чаще мы запрашиваем координаты, тем чаще "пробуждаем" процессор и радиомодуль.

3. Задержка (Latency):
   - Чем быстрее нам нужен результат, тем меньше у системы возможностей для оптимизации и объединения запросов.

Три кита расхода батареи

Ваша главная задача как разработчика — найти правильный баланс между этими тремя параметрами для каждой конкретной функции.

Оптимизация №1: Запуск по активности (ActivityRecognition)

Идея: Не запрашивать геолокацию постоянно, а включать ее только тогда, когда пользователь совершает нужное действие (например, едет на велосипеде).

Как работает:

1. Вы подписываетесь на изменения активности пользователя (ходьба, в транспорте, стоит на месте).

2. Когда система обнаруживает начало нужной активности (ACTIVITY_TRANSITION_ENTER), вы запускаете requestLocationUpdates().

3. Когда активность завершается (ACTIVITY_TRANSITION_EXIT), вы останавливаете requestLocationUpdates().

Оптимизация №1: Запуск по активности (ActivityRecognition)

Требует разрешения в манифесте:

```
<uses-permission android:name="com.google.android.gms.permission.ACTIVITY_RECOGNITION"/>
```

Пример: Идеально для фитнес-трекера. Запись трека начинается автоматически, как только пользователь побежал, и останавливается, когда он перешел на шаг.

Оптимизация №2: Пакетная доставка (setMaxWaitTime)

Это ключевая оптимизация для любой фоновой работы. Она позволяет системе «копить» точки и доставлять их пачкой, вместо того чтобы будить телефон для каждой.

Оптимизация №2: Пакетная доставка (setMaxWaitTime)

Пример:

```
locationRequest.apply {
    // Хотим получать данные примерно каждые 10 минут
    interval = 10 * 60 * 1000

    // Но готовы подождать и получить их пачкой не реже, чем раз в час
    setMaxWaitTime(60 * 60 * 1000)

    priority = LocationRequest.PRIORITY_BALANCED_POWER_ACCURACY
}
```

Оптимизация №2: Пакетная доставка (setMaxWaitTime)

Как это работает:

- Без setMaxWaitTime: Телефон просыпается каждые 10 минут, чтобы доставить 1 точку. (6 пробуждений в час).
- С setMaxWaitTime: Телефон просыпается 1 раз в час и сразу доставляет ~6 накопленных точек. Экономия батареи — колоссальная.

Оптимизация №3: Пассивный режим

Когда использовать: Когда вам нужно «бесплатно» получать обновления, если кто-то другой их уже запросил. Идеально для виджетов или некритичных фоновых обновлений.

Как это работает:

- Ваше приложение не инициирует запросы геолокации.
- Оно просто «слушает» эфир: если Google Карты или другое приложение запросили координаты, система пришлет их и вам.

Оптимизация №3: Пассивный режим

Пример:

```
locationRequest.apply {
    priority = LocationRequest.PRIORITY_NO_POWER

    // Указываем, как часто мы готовы обрабатывать "пойманные" обновления
    interval = 15 * 60 * 1000 // например, не чаще раза в 15 минут
}
```

Оптимизация №3: Пассивный режим

Плюсы: Практически нулевой расход батареи.

Минусы: Нет никаких гарантий, что вы вообще получите обновления. Если пользователь не открывает Карты, ваше приложение может не получать данные часами.

Чек-лист: Как не разрядить батарею

Запрашивать минимально необходимую точность. Не используйте HIGH_ACCURACY, если вам достаточно знать город.

Всегда отключать обновления, когда они не нужны. Интегрируйте removeLocationUpdates() с жизненным циклом Activity/Fragment/Service.

Предпочитать getLastLocation() и Geofencing API подписке на обновления, если они решают вашу задачу.

Использовать службу переднего плана (Foreground Service) для долгих задач, которые инициировал пользователь (например, запись трека). Это честно информирует пользователя о том, что приложение работает и тратит энергию.

Устанавливать таймауты (setExpirationDuration) на запросы. Это защитит от «вечных» обновлений, которые могут возникнуть из-за ошибок в коде.

Тестирование геолокации

Как протестировать новую систему разрешений

С появлением выбора точности, ваше приложение обязательно нужно протестировать в трех сценариях.

Сценарий 1: Пользователь изначально выбрал ПРИБЛИЗИТЕЛЬНУЮ точность.

1. Запросите FINE + COARSE.
2. В системном диалоге выберите «Приблизительно».
3. Проверка: Ваша функция работает корректно? Не падает ли приложение? Отображается ли заглушка или альтернативный контент?

Как протестировать новую систему разрешений

С появлением выбора точности, ваше приложение обязательно нужно протестировать в трех сценариях.

Сценарий 2: Пользователь ПОНИЗИЛ права в настройках.

1. Предоставьте приложению «Точное» местоположение.
2. Перейдите в Настройки системы -> Приложения -> [Ваше приложение] -> Разрешения -> Местоположение.
3. Отключите переключатель «Использовать точное местоположение».
4. Проверка: Система перезапустит процесс вашего приложения. Убедитесь, что после перезапуска оно корректно работает с приблизительными данными.

Как протестировать новую систему разрешений

С появлением выбора точности, ваше приложение обязательно нужно протестировать в трех сценариях.

Сценарий 3: Пользователь ПОВЫСИЛ права в настройках.

1. Предоставьте приложению **«Приблизительное»** местоположение.
2. Перейдите в настройки и **включите** «Использовать точное местоположение».
3. **Проверка:** Приложение (без перезапуска) начало получать более точные данные?

Визуализация данных: Maps SDK

Полученные координаты можно и нужно визуализировать. Для этого используется Maps SDK for Android.

Основные возможности:

- Добавить карту в приложение: Встраивается как Fragment прямо в ваш XML-макет.
- Поставить маркер: Отметить на карте конкретную точку (например, текущее положение пользователя).
- Кастомизация: Можно менять вид карты (спутник, рельеф), рисовать фигуры (круги, полилинии для маршрутов), менять иконки маркеров и многое другое.
- Управление камерой: Программно перемещать «взгляд» пользователя на нужную точку.

Визуализация данных: Maps SDK

Пример:

```
val userLocation = LatLng(location.latitude, location.longitude)
googleMap.addMarker(
    MarkerOptions()
        .position(userLocation)
        .title("Я здесь!")
)
googleMap.moveCamera(CameraUpdateFactory.newLatLng(userLocation))
```

Современный подход с Coroutines & Flow

Вместо использования LocationCallback вручную, можно обернуть подписку в callbackFlow.

Преимущества:

- Безопасность: Поток автоматически отменит подписку на обновления, когда корутина, в которой он запущен, завершится (например, при уходе с экрана). Это решает проблему «забытых» removeLocationUpdates().
- Интеграция с ViewModel: Идеально вписывается в современную архитектуру и lifecycleScope.
- Богатый API: Можно использовать все операторы Flow (debounce, filter, map и т.д.) для обработки потока координат.

Современный подход с Coroutines & Flow

Как это выглядит (концептуально):

```
fun getLocationsFlow(locationRequest: LocationRequest): Flow<Location> = callbackFlow {
    val locationCallback = object : LocationCallback() {
        override fun onLocationResult(result: LocationResult) {
            trySend(result.lastLocation) // Отправляем в Flow
        }
    }
    // При старте Flow
    fusedLocationClient.requestLocationUpdates(locationRequest, locationCallback,
Looper.getMainLooper())
    // При отмене Flow
    awaitClose {
        fusedLocationClient.removeLocationUpdates(locationCallback)
    }
}
```

Итоги

1. Всегда думайте о разрешениях: Различайте точность и контекст, используйте современный API для запросов.
2. Выбирайте правильный инструмент: getLastLocation, requestLocationUpdates или Geofencing в зависимости от задачи.
3. Оптимизируйте расход батареи: Настраивайте priority, interval и всегда используйте batching для фона.
4. Тестируйте сценарии смены разрешений.
5. Визуализируйте результат с помощью Maps SDK.